ARQUITECTURA DE AGENTES IA · UTDT 2026

Módulo 6 Conocimiento y Memoria

Memoria · RAG moderno · Agentic RAG · Grafos de conocimiento · Neo4j y Cypher · ADK2

UTDT 2026 | MÓDULO 6 DE 13

El agente que no recuerda no aprende

> *"La diferencia entre un agente útil y uno brillante no está en el modelo — está en qué conocimiento puede recuperar y qué historia puede recordar."*

Un agente opera en dos dimensiones de información:

- Conocimiento — hechos externos recuperados en el momento de la generación
- Memoria — el historial propio del agente: intercambios previos, salidas de herramientas, actualizaciones de estado

Ingeniería de contexto (Módulo 5) = seleccionar y ensamblar el conocimiento + la memoria correctos dentro de la ventana de contexto.

El módulo de hoy cubre cómo construir, recuperar y mantener ambas dimensiones a escala de producción.

Lo que veremos hoy

MEMORIA DEL AGENTE

- Tipos: working, episódica, semántica, procedural
- Ciclo de vida: escribir, leer, actualizar, olvidar
- Los 4 niveles de mecanismo (y qué es FIFO)
- Session y Memory en ADK2

RAG Y SU EVOLUCIÓN

- RAG = *Retrieval-Augmented Generation*: generar con información recuperada
- Chunking, índices, metadata y ACL
- Advanced RAG: híbrido, RRF, reranking
- Agentic RAG y su diagrama de flujo

GRAFOS DE CONOCIMIENTO

- La intuición y qué preguntas resuelven que el RAG no
- Anatomía: nodos, relaciones, propiedades, esquema
- Neo4j y Cypher, paso a paso
- Agente + grafo; ejemplos de fraude y normas

Regla de oro: elegir el nivel de complejidad más bajo que resuelva el problema. Las siglas se definen al usarlas y en las slides de siglas del final.

Conocimiento vs. Memoria — La distinción clave

CONOCIMIENTO

- Hechos externos al agente
- Se recuperan *en tiempo de generación*
- Verificables y actualizables
- Fuente: documentos, APIs, bases de datos
- Mecanismo: RAG, GraphRAG
- Riesgo principal: *stale data* (datos obsoletos)

MEMORIA

- Historia propia del agente
- Intercambios previos, salidas de herramientas
- Continuidad entre turnos y sesiones
- Fuente: el agente mismo
- Mecanismo: jerarquía de memoria
- Riesgo principal: *context drop* (pérdida silenciosa)

> El criterio es el origen y el ciclo de vida, no la tecnología. Un vector store o un grafo pueden alojar cualquiera de los dos: es conocimiento si lo escribe una ingesta curada desde fuentes externas y es compartido; es memoria si lo produce el agente al operar y es propio de un usuario. Mismo motor, colecciones separadas.

> La ingeniería de contexto es el arte de ensamblar la combinación óptima de ambos en cada llamada al modelo.

Cuatro tipos de memoria — qué se recuerda

Working

de trabajo

Qué guarda
Lo que el agente tiene «a la vista» en esta llamada.

Dónde vive
Ventana de contexto: prompt, historial reciente, resultados de tools.

Ejemplo
Los últimos mensajes de la conversación.

Episódica

Qué guarda
Experiencias concretas: qué pasó, cuándo y con quién.

Dónde vive
Log de interacciones; vector store de sesiones pasadas.

Ejemplo
El reclamo del cliente de la semana pasada.

Semántica

Qué guarda
Hechos y conocimiento desligados del episodio.

Dónde vive
Perfil de usuario y hechos consolidados de sus sesiones; a veces, un grafo de sus entidades.

Ejemplo
«El cliente opera en pesos y prefiere español».

Procedural

Qué guarda
Cómo hacer las cosas: reglas, habilidades, flujos.

Dónde vive
System prompt, instrucciones, código de las tools, skills.

Ejemplo
El protocolo de escalamiento de un caso.

Tres ejes distintos, que no hay que mezclar: el tipo (qué se recuerda), el mecanismo (cómo se recupera: los 4 niveles) y el alcance (una sesión o entre sesiones). Origen: psicología cognitiva (Tulving; Atkinson–Shiffrin), adaptada a agentes por el marco CoALA (Sumers et al., 2023).

La memoria no es un depósito: es un ciclo de cinco operaciones



¿Qué vale la pena guardar y cuándo? Hechos, no transcripciones

Tipo de memoria, metadata y dueño (usuario, app) Filtros de acceso

Puntaje = relevancia + recencia + importancia (top-k con filtro)

El dato nuevo contradice al viejo: ¿reemplazar, versionar o pedir confirmación?

TTL (tiempo de vida), decaimiento por desuso, consolidación y derecho al olvido (trazable)

Diferencia con RAG: el corpus de RAG se ingesta y se revisa offline; la memoria se escribe mientras el agente opera. Por eso necesita políticas explícitas de actualización y de olvido.

Las decisiones difíciles no están en «almacenar» sino en qué escribir, cómo puntuar al leer y cuándo olvidar. La lectura suele combinar tres señales: relevancia semántica, recencia e importancia.

Los 4 niveles — de simple a complejo

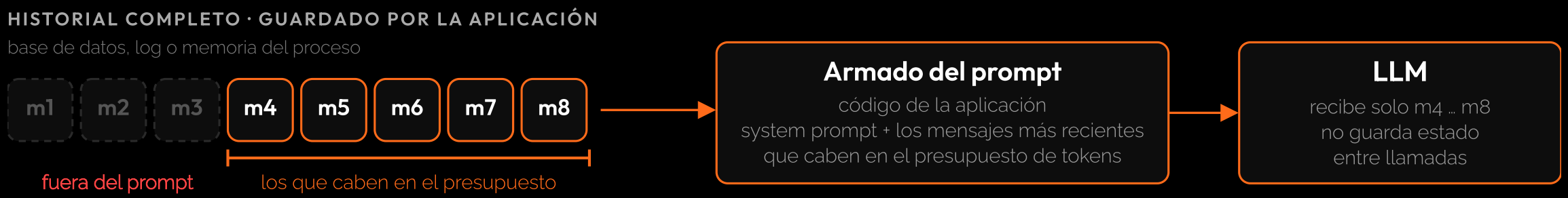
1 Ventana deslizante (FIFO: First In, First Out) — La aplicación reenvía al modelo solo los mensajes más recientes que caben en el presupuesto de tokens; los más antiguos salen primero del prompt. No lo hace el modelo ni borra el historial guardado. Riesgo: contexto importante cae silenciosamente.

2 BM25 full-text search — Índice invertido + scoring BM25. Coincidencia exacta de palabras clave. Rápido. No captura paráfrasis.

3 Memoria semántica (vector store) — Embeds de interacciones → FAISS/VectorDB → búsqueda por similitud. Captura el significado, tolera paráfrasis.

4 Memoria semántica experiencial — Híbrido: búsqueda vectorial sobre interacciones pasadas + ventana de contexto reciente. Continuidad *entre sesiones*.

FIFO — quién descarta, dónde y qué se pierde de verdad



Turno siguiente: llega m9 → la ventana se corre → m4 sale del prompt.

Qué significa

First In, First Out: el primero que entró es el primero que sale, como una fila. Es una política de descarte para buffers de tamaño limitado (colas, cachés); acá se aplica al historial de la conversación.

Quién lo hace

La aplicación, cuando arma el prompt de cada llamada. El LLM no guarda estado entre llamadas: solo ve lo que se le envía. Lo descartado puede seguir guardado; simplemente no llega al modelo.

Dónde falla

El criterio es la antigüedad, no la importancia: si el diagnóstico inicial está en m1, sale primero. Alternativas: resumir lo antiguo, fijar mensajes clave o recuperar por similitud (niveles 2 a 4).

BM25 — búsqueda por palabras clave

CORPUS (4 DOCUMENTOS)

- **d1 · El legajo del cliente está vencido**
- **d2 · Actualizar el legajo del cliente**
- **d3 · El límite de crédito del cliente**
- **d4 · La tasa del crédito**

ÍNDICE INVERTIDO: PALABRA → DOCUMENTOS

vencido	d1	rara: pesa mucho
legajo	d1 d2	intermedia
cliente	d1 d2 d3	común: pesa poco
el · del	d1 d2 d3	casi en todos: ≈ 0

CONSULTA: «legajo vencido»

- **1 · d1** — tiene las dos palabras
- **2 · d2** — solo «legajo»
- **3 · d3, d4** — ninguna palabra

puntaje = suma del peso de cada palabra que coincide con la consulta

De TF-IDF a BM25

Inspirado en TF-IDF: cuántas veces aparece una palabra en el documento (TF) × qué tan rara es en el corpus (IDF). Agrega dos ajustes: repetirla no suma sin límite y un documento largo no gana por serlo.

Cuándo conviene

Cuando importan las palabras exactas: códigos, siglas, nombres propios, logs. Es el ranking por defecto de Elasticsearch y OpenSearch.

Dónde falla

No entiende sinónimos ni paráfrasis: «caducado» no encuentra «vencido». Tampoco conoce el orden de las palabras ni el contexto.

Memoria semántica — búsqueda por significado

Recuerdo
texto

→

Modelo de embeddings
el mismo para guardar y consultar

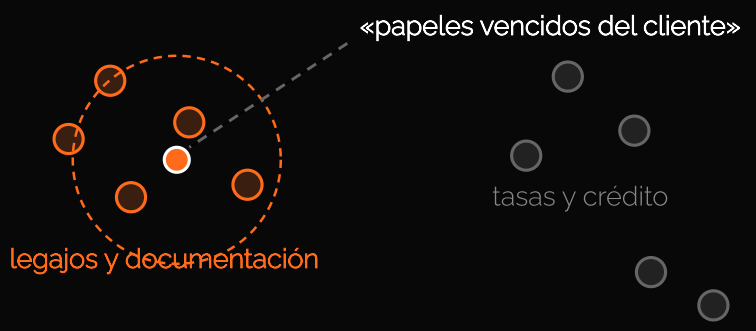
→

Vector
[0,12; −0,03; 0,58; …]

→

Índice vectorial
búsqueda de vecinos (ANN)

ESPACIO DE SIGNIFICADO (PROYECCIÓN)



Cercanía = similitud de significado

Se mide con el coseno: el ángulo entre dos vectores. Misma dirección → mismo sentido → textos parecidos.

«papeles vencidos del cliente» queda cerca de «legajo desactualizado» aunque no compartan ninguna palabra: eso es tolerar la paráfrasis.

Qué es

Un modelo de embeddings convierte cada texto en un vector (p. ej., 384 o 768 números): textos parecidos quedan cerca. La consulta se codifica igual y se buscan los vecinos más cercanos.

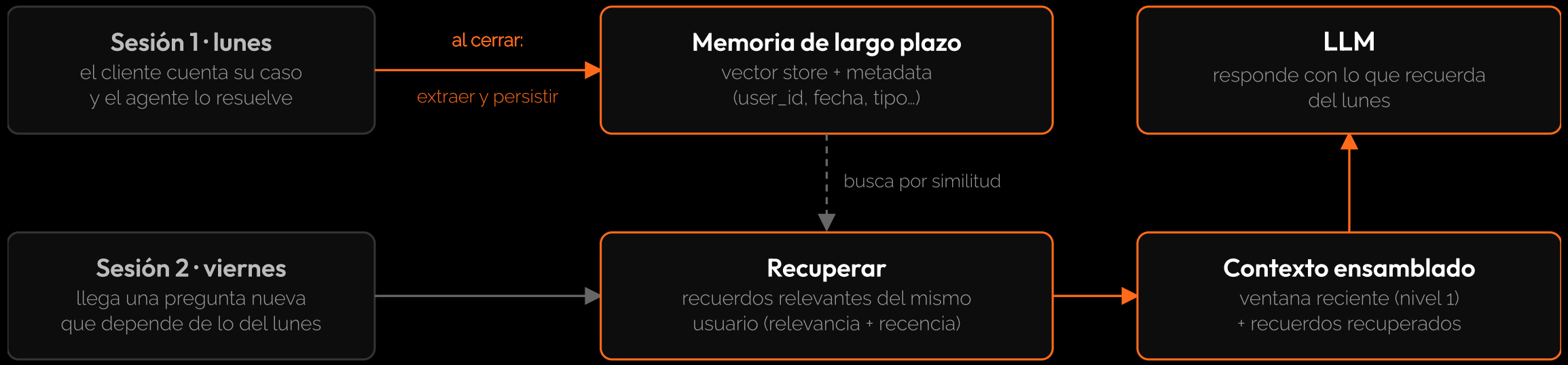
Cuándo conviene

Lenguaje libre: paráfrasis, sinónimos, varios idiomas; hechos que el usuario expresa de muchas maneras.

Dónde falla

Códigos y siglas exactas se diluyen; no explica por qué devolvió algo; si cambia el modelo hay que reembeber todo; por sí solo no persiste entre sesiones.

Memoria experiencial — recordar entre sesiones



Qué agrega

Al nivel 3 le suma persistencia y una separación clara: la ventana reciente (corto plazo) más una memoria de largo plazo consultada por similitud. Es lo que permite «recordar lo del lunes».

Decisiones de diseño

Qué persistir (episodios o hechos consolidados), cómo recuperar (relevancia + recencia + importancia) y cómo aislar a cada usuario. En ADK2: `add_session_to_memory` y `search_memory`.

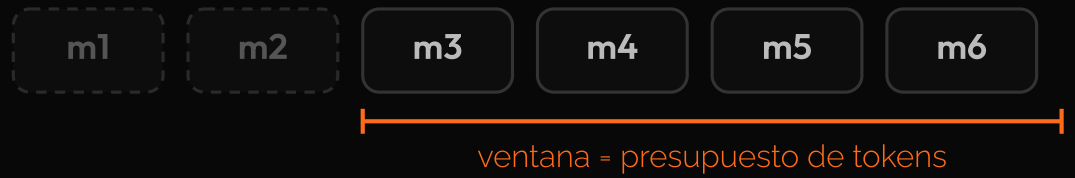
Riesgos

Persistir todo genera ruido; los recuerdos se vuelven obsoletos o se contradicen; y es dato personal: exige retención y derecho al olvido.

Los cuatro mecanismos de un vistazo

NIVEL 1 · FIFO (FIRST IN, FIRST OUT)

salen primero del prompt



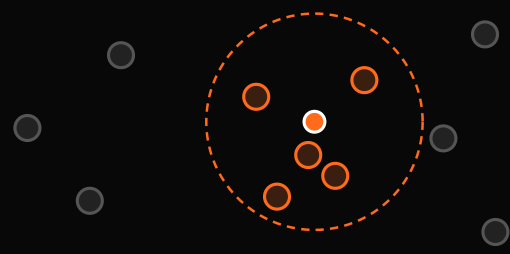
El código que arma el prompt deja afuera m1 y m2:
siguen guardados, pero el modelo ya no los ve.

NIVEL 2 · BM25 (BEST MATCHING 25)



Coincidencia exacta: no une «vencido» con «desactualizado».

NIVEL 3 · MEMORIA SEMÁNTICA (VECTORES)

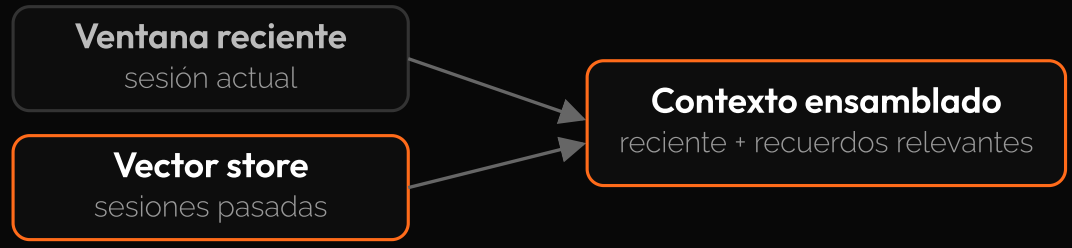


Cada texto = un vector

Cercanía = similitud de sentido
«vencido» ≈ «desactualizado»
Tolera paráfrasis

● consulta ○ vecinos más cercanos

NIVEL 4 · EXPERIENCIAL (ENTRE SESIONES)



Al cerrar la sesión se persiste: continuidad entre sesiones.

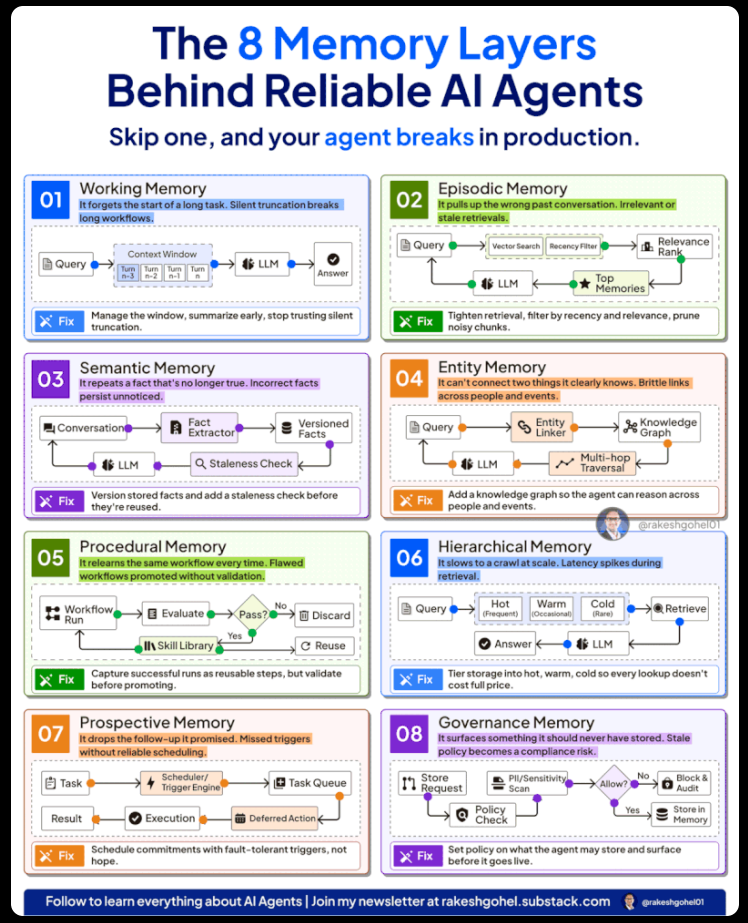
Cada nivel resuelve una limitación del anterior. El código de cada uno se ve en las demos.

¿Qué memoria elegir?

ESCENARIO	NIVEL RECOMENDADO
Chatbot simple, sesiones cortas	Ventana deslizante
Recuperación por términos exactos	BM25 full-text
Similitud semántica, paráfrasis	Vector store (FAISS)
Continuidad entre sesiones	Memoria experiencial
Preguntas multi-salto, relaciones	GraphRAG (Neo4j)
Base de conocimiento estática y grande	Long-context O RAG híbrido
Conocimiento con actualización en tiempo real	Grafo dinámico *(alta complejidad)*

> *Regla: escalar al nivel siguiente solo cuando el nivel actual falla.*

8 capas de memoria vs. los 4 niveles del módulo



Infografía: Rakesh Gohel (@rakeshgohel01)

01 Working — Nivel 1 (FIFO). El «silent truncation» es el anti-pattern de la ventana deslizante.

02 Episodic — Nivel 4 (experiencial). Falta el filtro por recencia: el código hace solo top-k por similitud.

03 Semantic — Nivel 3 (vector store de memoria, no el corpus del RAG). Faltan versionar hechos y revisar vigencia; ver «embeddings obsoletos».

04 Entity — Grafo de entidades. Como memoria: lo que el agente sabe del usuario; como conocimiento: GraphRAG sobre un corpus.

05 Procedural — No cubierta en este módulo.

06 Hierarchical — No cubierta: tiering hot / warm / cold.

07 Prospective — No cubierta: disparadores y tareas diferidas.

08 Governance — Los 5 controles de la Slide 65 (PII, retención, trazabilidad).

Los 4 niveles ordenan mecanismos de recuperación; las 8 capas ordenan modos de falla. Se complementan.

Pipeline RAG — Arquitectura de dos fases

FASE 1 · INDEXACIÓN (OFFLINE)

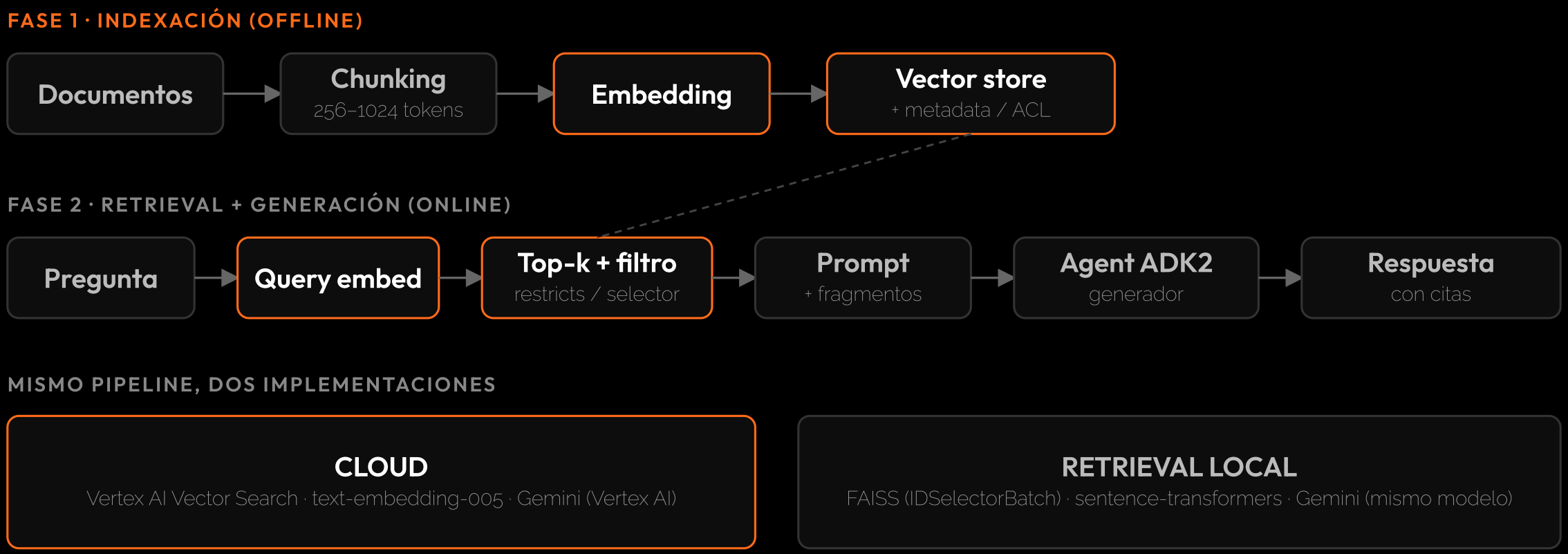
1. Chunking — Dividir documentos en fragmentos (256–1024 tokens)
2. Embedding — Codificar cada chunk en un vector
3. Almacenamiento — Insertar en vector DB con metadata

FASE 2 · RETRIEVAL + GENERACIÓN (ONLINE)

1. Query embed — Codificar la pregunta del usuario
2. Top-k retrieval — Buscar chunks más similares
3. Prompt injection — Inyectar chunks en el prompt
4. Generación — El modelo responde con contexto

Hallazgo clave: RAG híbrido + recuperación selectiva supera al long-context puro en casos de uso enterprise críticos por precisión

Pipeline RAG — dónde cambia cloud vs. local



Cambian los cuadros resaltados (embeddings, vector store y filtro); el modelo, el flujo y el `Workflow` de ADK2 son idénticos.

Chunking — cómo se corta define qué se encuentra

ESTRATEGIA	CÓMO CORTA	CUÁNDO CONVIENE	RIESGO
Tamaño fijo + solape	Cada N tokens, con solape entre cortes	Texto homogéneo; punto de partida	Corta ideas por la mitad
Recursivo / por estructura	Respeta títulos, párrafos, artículos	Normativa, manuales, contratos	Requiere documentos bien estructurados
Semántico	Corta donde cambia el tema (por similitud)	Texto largo sin estructura clara	Más costoso de indexar; tamaños irregulares
Parent–child	Busca con fragmentos chicos; devuelve el bloque padre	Precisión al buscar y contexto al responder	Más complejidad en el índice

Tensión central: fragmentos chicos → búsqueda precisa pero contexto pobre; fragmentos grandes → contexto rico pero búsqueda diluida. Se resuelve midiendo sobre preguntas reales, no eligiendo un tamaño «estándar» (256–1024 tokens es solo el rango habitual).

Índices vectoriales — de la búsqueda exacta a la aproximada

ÍNDICE	QUÉ SIGNIFICA	IDEA	TRADE-OFF
Flat	«plano»: sin estructura	Compara la consulta contra todos los vectores	Exacto; costo lineal
IVF	*Inverted File* (archivo invertido)	Agrupa los vectores en celdas (k-means) y busca solo en las cercanas	Rápido; puede perder vecinos de otras celdas
HNSW	*Hierarchical Navigable Small World*	Grafo de vecinos en capas: se navega de lo grueso a lo fino	Muy rápido y preciso; más memoria
ScaNN	*Scalable Nearest Neighbors* (Google)	Cuantización + búsqueda aproximada; base de Vertex AI Vector Search	Escala a miles de millones

ANN (*Approximate Nearest Neighbors*, vecinos más cercanos aproximados) intercambia una pequeña pérdida de exactitud por una gran ganancia de velocidad. FAISS (*Facebook AI Similarity Search*) es una biblioteca que implementa estos índices. La búsqueda se filtra *durante* el recorrido con la metadata del fragmento (Slides 19–20).

Qué se guarda con cada vector — la metadata

> Un fragmento indexado = vector + texto + metadata. El vector sirve *solo para buscar*, todo lo demás sirve para filtrar, citar, auditar y mantener el índice.

CAMPO	EJEMPLO	PARA QUÉ SIRVE
`doc_id` · `chunk_id`	`politica-credito-v3` · `#c04`	Trazabilidad documento → fragmentos → vectores; actualizar o borrar un documento completo
`texto`	«El límite de asistencia a un grupo económico…»	Es lo que se inyecta en el prompt y se cita
fuente y ubicación	Política de Crédito v3 · pág. 12 · §4.2	Citas verificables y auditoría
fecha y versión	vigente desde 2026-03-01 · v3	Filtrar lo vigente; detectar datos obsoletos
tipo · idioma	normativa · es	Filtrar y rutear por tipo; elegir el modelo de embeddings
clasificación	pública · interna · confidencial	Política de datos: dónde y cómo puede almacenarse
`acl`	["riesgos", "auditoría"]	Quién puede leer este fragmento (próxima slide)
`user_id`	u-1042	En memoria del agente: de quién es el recuerdo; aísla usuarios
modelo de embeddings	text-embedding-005	Si cambia el modelo hay que reembeber todo

ACL — quién puede ver cada fragmento

ACL = *Access Control List*: la lista de los usuarios, grupos o roles autorizados a acceder a un recurso. En RAG, cada fragmento lleva la ACL del documento del que proviene.

FRAGMENTO	ACL DEL FRAGMENTO	ANA · rol: riesgos	BETO · rol: atención
Política de crédito corporativo	riesgos, auditoría	lo ve	no lo ve
Horarios de atención al público	todos	lo ve	lo ve
Informe de hallazgos de auditoría	auditoría	no lo ve	no lo ve

Filtro durante la búsqueda: ACL del fragmento ∩ roles del usuario ≠ ∅ → solo esos fragmentos compiten en el ranking

1 · Al indexar
El fragmento hereda la ACL del documento, tomada del sistema de origen (gestor documental, carpeta).

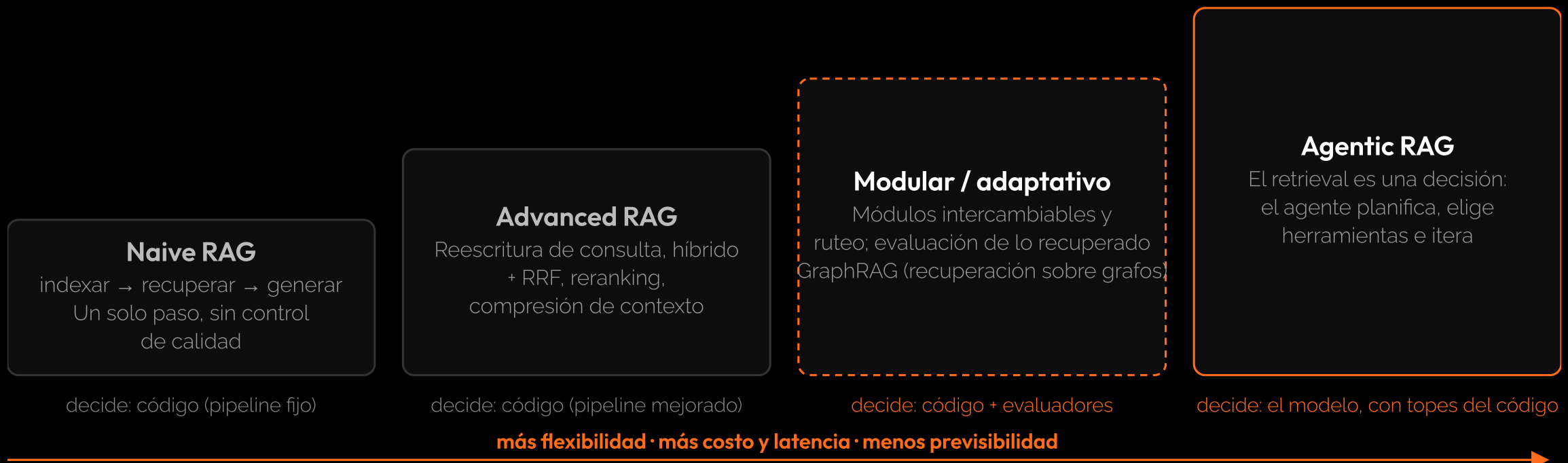
2 · Al consultar
Se identifica al usuario y sus grupos o roles (identidad corporativa).

3 · En la búsqueda
Solo compiten los fragmentos cuya ACL incluye algún rol del usuario.

Riesgo
Si los permisos cambian en el origen y no en el índice, se filtra con permisos viejos.

Evolución del RAG — de un pipeline fijo a un agente que decide

Clasificación didáctica: las categorías se superponen, no son una jerarquía estricta.



Cada escalón agrega capacidad y también costo, latencia y superficie de fallas. Las próximas slides recorren estos escalones.

Advanced RAG — mejorar antes, durante y después de recuperar



Las mejoras más rentables suelen ser dos: búsqueda híbrida (Slides 23–24) y reranking (Slide 25). HyDE: el modelo redacta una respuesta hipotética y se busca con su embedding, no con el de la pregunta.

RAG híbrido — por qué combinar BM25 y vectores

Consulta: «error E-4021 al cerrar el mes»

BM25 · VE LAS PALABRAS EXACTAS

1·A	Código E-4021: falla en el cierre	12,7
2·B	Problemas al finalizar el mes contable	6,1
3·C	Manual del módulo de cierre	4,0

puntaje BM25: sin tope, depende del corpus

VECTORES · VEN EL SIGNIFICADO

1·B	Problemas al finalizar el mes contable	0,83
2·C	Manual del módulo de cierre	0,78
3·D	Guía de cierre anual	0,74

similitud coseno: entre 0 y 1

Cada método ve algo que el otro no: A solo aparece en BM25 (por el código exacto); D solo en vectores.
Pero 12,7 y 0,83 no se pueden sumar: están en escalas distintas.

Con la misma consulta, cada método ve documentos distintos. La búsqueda híbrida las combina; el problema es cómo fusionar dos rankings cuyos puntajes no son comparables.

Fusión por rangos (RRF), paso a paso

1 · Ignorar los puntajes

Solo importa la posición de cada documento en cada lista. Así no hay que comparar 12,7 con 0,83.

2 · Cada lista «vota»

Un documento en la posición *p* recibe 1 ÷ (60 + p). La 1.ª vale 0,0164; la 2.ª, 0,0161; la 3.ª, 0,0159. Si no aparece, no recibe nada.

3 · Sumar y ordenar

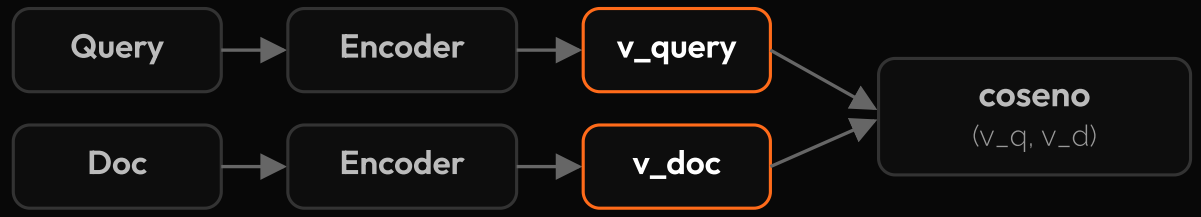
Se suman los votos de ambas listas y se ordena de mayor a menor. El 60 es la constante *k*.

DOCUMENTO	EN BM25	VOTO	EN VECTORES	VOTO	TOTAL RRF	PUESTO
B · Problemas al finalizar el mes	2.º	0,0161	1.º	0,0164	0,0325	1.º
C · Manual del módulo de cierre	3.º	0,0159	2.º	0,0161	0,0320	2.º
A · Código E-4021	1.º	0,0164	—	0	0,0164	3.º
D · Guía de cierre anual	—	0	3.º	0,0159	0,0159	4.º

Cómo leerlo: B no fue 1.º en BM25, pero *ambos* métodos lo consideran bueno, y gana. A fue 1.º en BM25 pero solo uno lo vio: queda 3.º. ¿Por qué 60? Con una constante chica, el 1.º valdría el doble que el 2.º; con 60 la diferencia entre posiciones vecinas es mínima y pesa más *coincidir* en las dos listas que ser primero en una. Alternativa: suma ponderada de puntajes normalizados, que exige normalizar escalas.

Reranking — bi-encoder para encontrar, cross-encoder para decidir

BI-ENCODER · RECUPERACIÓN (RÁPIDO, APROXIMADO)

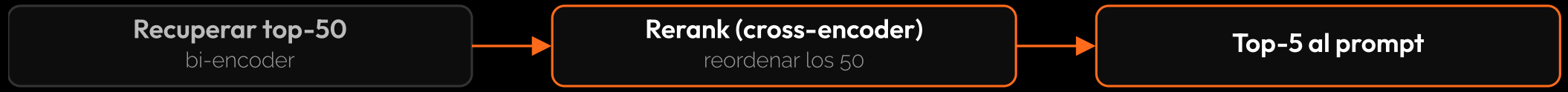


Los documentos se codifican una sola vez, offline.
Escala a millones. Pero query y documento
no se «ven» entre sí: menos preciso.

CROSS-ENCODER · RERANKING (LENTO, PRECISO)



Una pasada por cada par: caro.
Solo se aplica a unos pocos candidatos.
Ve pregunta y documento juntos: más preciso.



Patrón habitual: recuperar amplio y barato, reordenar estrecho y caro. El reranker corrige lo que la similitud de embeddings no distingue.

Contexto largo vs. RAG — ¿Cuándo usar cada uno?

MODELO	CONTEXTO MÁXIMO
Otros modelos de frontera	Cientos de miles de tokens (varía por versión y proveedor)
Gemini 2.5	hasta 1M tokens (~750K palabras, ~2.500 páginas)

Usar Long-Context cuando...

- El documento cabe entero (~<500 páginas)
- Se necesita razonamiento sobre el *todo*
- Latencia no es crítica

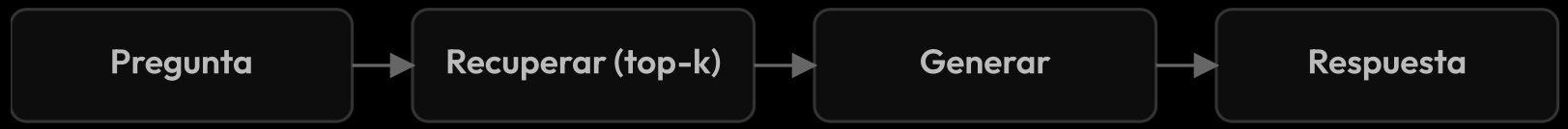
Usar RAG cuando...

- Corpus de millones de documentos
- Precisión factual es crítica
- Actualización frecuente del corpus

> *Además: el contexto largo no tiene control de acceso. Si el corpus entero está en el prompt, todos los usuarios «ven» todo (Slide 65).*

Agentic RAG — la recuperación pasa a ser una decisión del agente

RAG CLÁSICO · PIPELINE FIJO



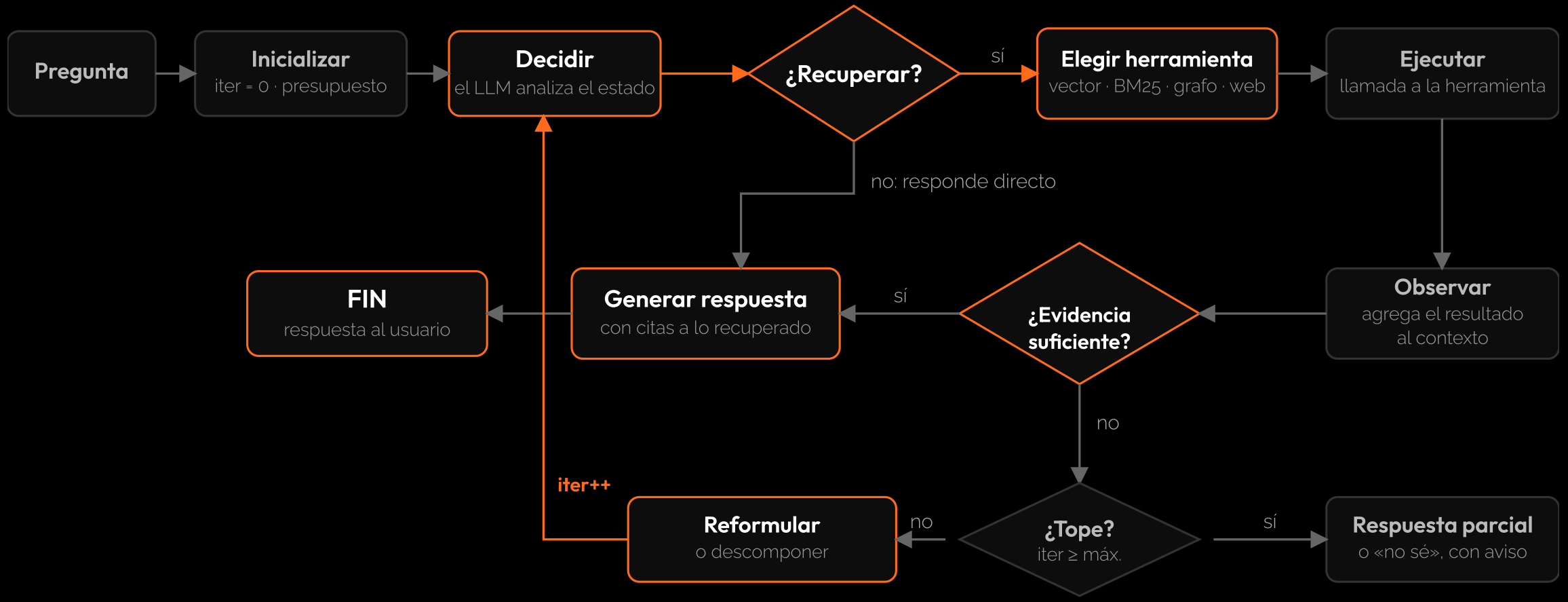
El código decide:
siempre recuperar, una sola vez.

AGENTIC RAG · BUCLE DE DECISIÓN



En el pipeline fijo, el código decide siempre recuperar una vez. En Agentic RAG, el modelo decide si recuperar, dónde, cuántas veces y cuándo parar; el código le pone los límites.

Agentic RAG — diagrama de flujo



Naranja = decisión del modelo. Gris = lo fija el código: contadores, herramientas disponibles, tope de iteraciones y presupuesto. Las tres salidas posibles: responder directo, responder con evidencia, o responder parcialmente con aviso.

Agentic RAG — patrones habituales y cómo se controla

PATRONES

- Agente único con herramientas — un LLM elige entre vector, grafo, web…
- Planificación — descompone la pregunta en sub-preguntas y las resuelve en orden
- ReAct — alterna razonamiento y acción, observando cada resultado
- Multiagente — un agente por fuente o dominio, con un coordinador

RIESGOS

- Bucles — re-buscar lo mismo sin avanzar
- Costo y latencia variables: N llamadas al modelo por pregunta
- No determinismo — la misma pregunta puede seguir caminos distintos
- Parámetros que elige el modelo (saltos, k): deben acotarse en código

Controles: tope de iteraciones · tope de llamadas al modelo · herramientas con parámetros acotados · trazas de cada paso · evaluar con un conjunto de preguntas de referencia. *El modelo propone; el código dispone.*

Agentic RAG y ruteo en ADK2

Workflow: ruteo por tipo de pregunta

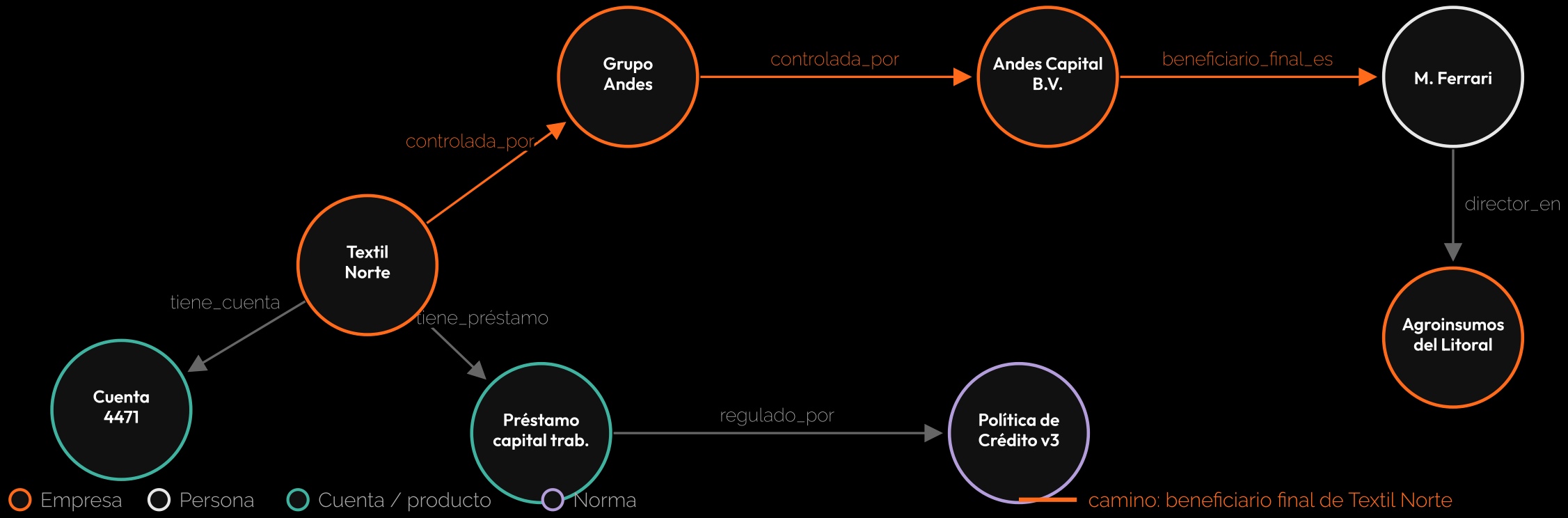
```
Workflow(name="enrutar", edges=[
    (START, clasificar),
    (clasificar, {"relacional": consultar_grafo,
                  "textual":    buscar_vectorial}),
    (consultar_grafo,  sintetizar),
    (buscar_vectorial, sintetizar),
])
# clasificar emite Event(route=...)
# con "relacional" o "textual"
```

Agent: el modelo elige la herramienta

```
agente = Agent(
    name="agente_rag",
    model="gemini-2.5-flash",
    instruction="Decidí si necesitás...",
    tools=[buscar_normativa,
           consultar_grafo,
           sumar_exposiciones])
# el modelo propone;
# el código pone el techo
RunConfig(max_llm_calls=6)
```

Criterio: si los caminos posibles se pueden enumerar de antemano, un `Workflow` es más predecible y auditable; si dependen de lo que se encuentre, un `Agent` con herramientas, siempre con tope. En el segundo caso, cada herramienta acota sus propios parámetros (saltos, cantidad de resultados) y las sumas las hace el código, no el modelo. Esquemas ilustrativos: el código ejecutable está en las demos.

Grafos de conocimiento — la intuición



Un grafo de conocimiento es un mapa de cosas y de cómo se relacionan: como el mapa del subte, una red social o un árbol genealógico. Lo valioso no son solo las cosas (los nodos) sino las conexiones (las relaciones). Preguntar es recorrer el mapa: el camino naranja responde una pregunta que ningún documento contesta por sí solo.

Cuando el RAG chunked falla — GraphRAG

RAG convencional falla en:

- Preguntas multi-salto: "¿Cuál es la empresa matriz del proveedor de Beyoncé?"
- Cadenas de relaciones que cruzan múltiples documentos
- Entidades que no aparecen en un solo chunk

GraphRAG extiende RAG con un grafo de conocimiento de entidades y relaciones:

Formato Triples

sujeto — predicado — objeto

```
"Jay-Z, cónyuge de, Beyoncé"
"Beyoncé, artista en, Columbia"
"Columbia, subsidiaria de, Sony"
```

3 Componentes

1. Grafo de conocimiento (Neo4j)
2. Sistema de recuperación
3. Modelo generativo

Preguntas que un grafo responde y un RAG no

TIPO DE PREGUNTA	EJEMPLO	POR QUÉ FALLA EL RAG POR FRAGMENTOS	QUÉ HACE EL GRAFO
Cadena (multi-salto)	«¿Quién es el beneficiario final de Textil Norte?»	La respuesta está repartida en varios documentos; ninguno la contiene	Recorre las relaciones encadenadas
Agregación	«¿Qué exposición total tiene el grupo económico?»	El top-k trunca: no ve a todos los miembros y no suma de forma confiable	Enumera todo el subgrafo y suma
Conectividad	«¿Cómo se relaciona el cliente A con el cliente B?»	No hay un texto que lo diga	Camino más corto entre ambos
Patrones y ciclos	«¿Qué cuentas comparten dispositivo con una marcada como fraude?»	El patrón no está escrito: emerge de la estructura	Busca el patrón en el grafo
Ausencia	«¿Qué normas siguen vigentes, o sea, no fueron reemplazadas?»	Recupera lo que existe; no puede probar que algo no existe	Filtra por no existencia de una relación
Global	«¿Cuáles son los temas principales del corpus?»	Ningún fragmento resume todo el corpus	Resúmenes por comunidad del grafo

Un RAG por fragmentos responde si alguien ya escribió la respuesta en un texto. Un grafo la calcula a partir de las relaciones. Para preguntas simples sobre un texto, el RAG sigue siendo la mejor opción.

Del texto al grafo

REGISTRO MERCANTIL

«Textil Norte S.A. está controlada por Grupo Andes.»

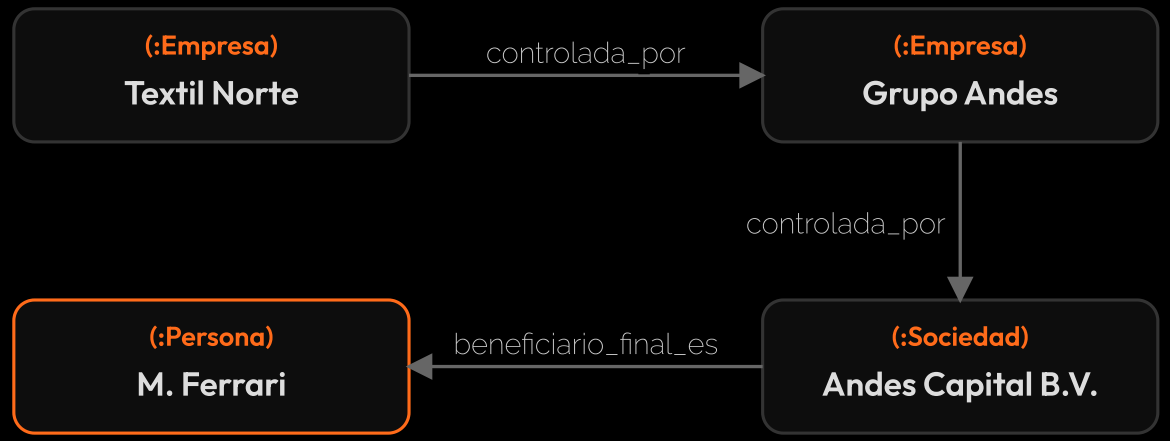
INFORME SOCIETARIO

«Grupo Andes S.A. pertenece a Andes Capital B.V.»

DECLARACIÓN DE BENEFICIARIOS

«El beneficiario final de Andes Capital B.V. es M. Ferrari.»

extraer



RAG POR FRAGMENTOS

La pregunta «¿quién es el beneficiario final de Textil Norte?» se parece al documento A (nombra a Textil Norte), no al C, que tiene la respuesta.

GRAFO DE CONOCIMIENTO

Recorre controlada_por → controlada_por → beneficiario_final_es y llega a M. Ferrari.
La respuesta se calcula, no se busca en un texto.

Al extraer entidades (nodos) y relaciones (aristas), aparece una cadena que nadie leería junta. «Grupo Andes» y «Grupo Andes S.A.» son la misma entidad: unificarlas es la resolución de entidades.

Anatomía de un grafo de conocimiento



1 · ETIQUETA
clasifica el nodo: Empresa, Persona, Sociedad

2 · PROPIEDADES
datos del nodo, como pares clave: valor

3 · RELACIÓN
une dos nodos; tiene un tipo y una dirección

4 · PROP. DE LA RELACIÓN
datos de la conexión: porcentaje, fecha, fuente

5 · ESQUEMA
qué etiquetas, tipos y propiedades son válidos

Valores ilustrativos.

Este es el modelo de property graph (grafo de propiedades): tanto los nodos como las relaciones llevan etiqueta o tipo y propiedades. Un camino es una secuencia alternada de nodos y relaciones; «Textil Norte → Grupo Andes → …» es un camino de longitud 2 o más.

RDF y ontologías — la otra familia, en una slide

TRIPLES RDF

RDF (*Resource Description Framework*) representa todo como triples sujeto – predicado – objeto, con identificadores globales (URI). Se consultan con SPARQL.

ONTOLOGÍA

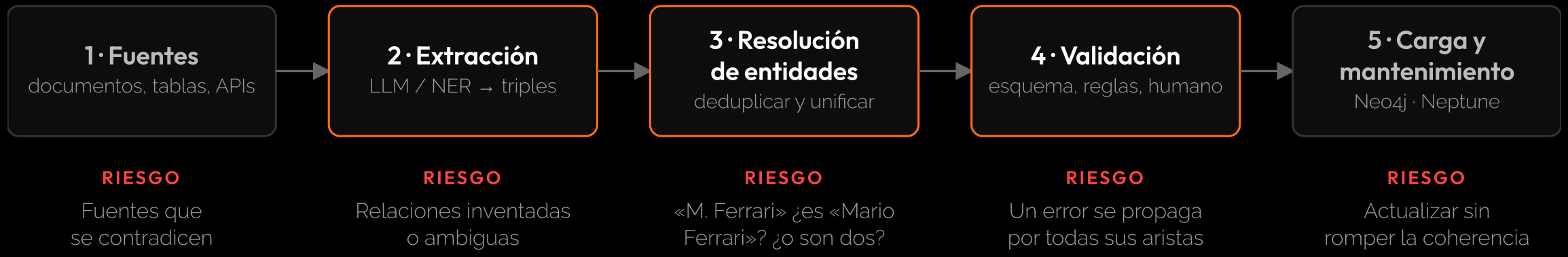
Un vocabulario formal: clases (Empresa, Banco), propiedades y jerarquías (un Banco es una Entidad Financiera). Se expresa en RDFS u OWL (*Web Ontology Language*), estándares del W3C.

RAZONAMIENTO LÓGICO

Con la ontología, un motor deduce hechos que nadie escribió: si «Banco X es un Banco» y «todo Banco es una Entidad Financiera», concluye que X es una Entidad Financiera. También con relaciones transitivas: si A controla a B y B controla a C, entonces A controla a C.

En este módulo trabajamos con property graph y Cypher (Neo4j): más directo para aplicaciones. RDF y ontologías se justifican cuando hay vocabularios compartidos entre organizaciones o se necesita razonamiento lógico formal. Amazon Neptune soporta ambos modelos.

Cómo se construye un grafo — y dónde se rompe



Indexar un grafo toma de horas a días, y las entidades críticas requieren revisión humana.
Es la razón por la que se justifica solo cuando el RAG por fragmentos falla.

La extracción con LLM convierte texto en triples, pero no es determinista: la misma frase puede producir relaciones distintas según el prompt. Por eso la validación contra el esquema es un paso obligatorio, no opcional.

Neo4j — una base de datos de grafos

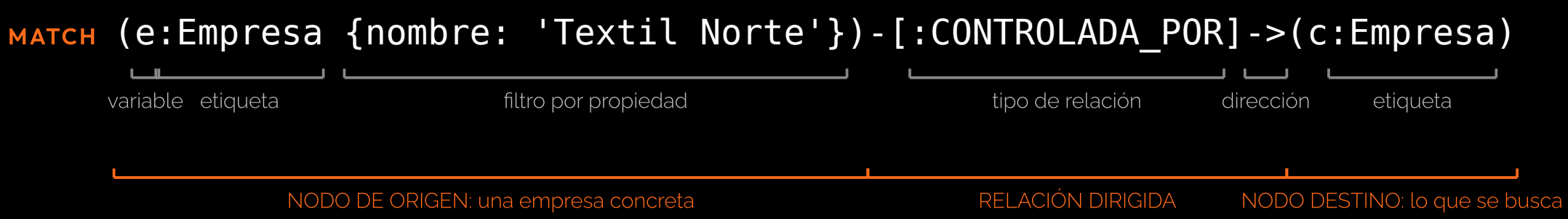
- Base de grafos nativa: guarda nodos y relaciones como datos de primera clase (modelo property graph).
- Cypher: su lenguaje de consulta, declarativo.
- Transacciones ACID (*Atomicity, Consistency, Isolation, Durability*): las escrituras son confiables.
- Conexión: protocolo *Bolt* (puerto 7687) con drivers oficiales (Python, Java, JavaScript…); navegador web en el puerto 7474.
- Índices: por propiedad, de texto completo (*fulltext*) y vectoriales: permite combinar búsqueda híbrida y recorrido en una sola base.
- Ediciones: Community (código abierto), Enterprise y Aura (servicio gestionado en la nube).
- En las demos: Neo4j Community en Docker (`docker compose up -d`).





No reemplaza a un buscador ni a una base relacional: conviene cuando las relaciones y los recorridos son el centro del problema.

Cypher — se dibuja el patrón que se busca



Cypher es declarativo: se dibuja el patrón que se busca, con «arte ASCII».
Nodos entre paréntesis ( ), relaciones entre corchetes [ ] con flecha; Neo4j decide cómo encontrarlo.

CLÁUSULA	QUÉ HACE	EJEMPLO
`MATCH`	Describe el patrón a encontrar	`MATCH (e:Empresa)-[:CONTROLADA_POR]->(c)`
`WHERE`	Filtra los resultados	`WHERE c.sector = 'holding'`
`RETURN`	Qué devolver	`RETURN e.nombre, c.nombre`
`ORDER BY` · `LIMIT`	Ordena y acota	`ORDER BY e.nombre LIMIT 10`

Cuatro preguntas sobre el grupo societario

1 · Un salto: ¿quién controla a Textil Norte?

```
MATCH (e:Empresa {nombre: 'Textil Norte'})
      -[:CONTROLADA_POR]->(c)
RETURN c.nombre
```

→ Grupo Andes

2 · Saltos variables: todo lo alcanzable hasta 3 saltos

```
MATCH p = (e:Empresa {nombre: $nombre})
   -[:CONTROLADA_POR|BENEFICIARIO_FINAL_ES*1..3]->(x)
RETURN x.nombre, length(p) AS saltos
```

→ Grupo Andes 1 · Andes Capital 2 · M. Ferrari 3

3 · Agregación: exposición del grupo

```
MATCH (e:Empresa {nombre: $nombre})
      -[:CONTROLADA_POR*0..3]->(g)
WITH DISTINCT g
MATCH (g)-[:TIENE_EXPOSICION]->(x:Exposicion)
RETURN sum(x.monto) AS exposicion
```

→ un número; `DISTINCT` evita contar dos veces

4 · Camino más corto entre dos entidades

```
MATCH p = shortestPath(
  (a:Persona {nombre: 'M. Ferrari'})
  -[*..6]-(b:Empresa {nombre: 'Textil Norte'}))
RETURN p
```

→ Ferrari – Andes Capital – Grupo Andes – Textil Norte

`*1..3` fija una cota de saltos: siempre conviene ponerla. `$nombre` es un parámetro. Los importes del ejemplo son ilustrativos.

Escribir en Neo4j — esquema, carga e índices

Restricciones e índices (se crean una vez)

```
CREATE CONSTRAINT entidad_nombre IF NOT EXISTS
  FOR (e:Entidad) REQUIRE e.nombre IS UNIQUE
CREATE FULLTEXT INDEX doc_ft IF NOT EXISTS
  FOR (d:Documento) ON EACH [d.texto]
CREATE VECTOR INDEX doc_vec IF NOT EXISTS
  FOR (d:Documento) ON d.embedding
  OPTIONS {indexConfig: {`vector.dimensions`: 768,
           `vector.similarity_function`: 'cosine'}}
```

Carga idempotente de una relación

```
MERGE (a:Entidad {nombre: $s})
MERGE (b:Entidad {nombre: $o})
MERGE (a)-[:CONTROLADA_POR]->(b)
```

CREATE vs. MERGE

`CREATE` siempre crea: correrlo dos veces duplica. `MERGE` busca y, si no existe, crea: es idempotente y se puede repetir sin miedo.

Restricción de unicidad

Impide dos nodos `:Entidad` con el mismo nombre; además acelera la búsqueda por nombre.

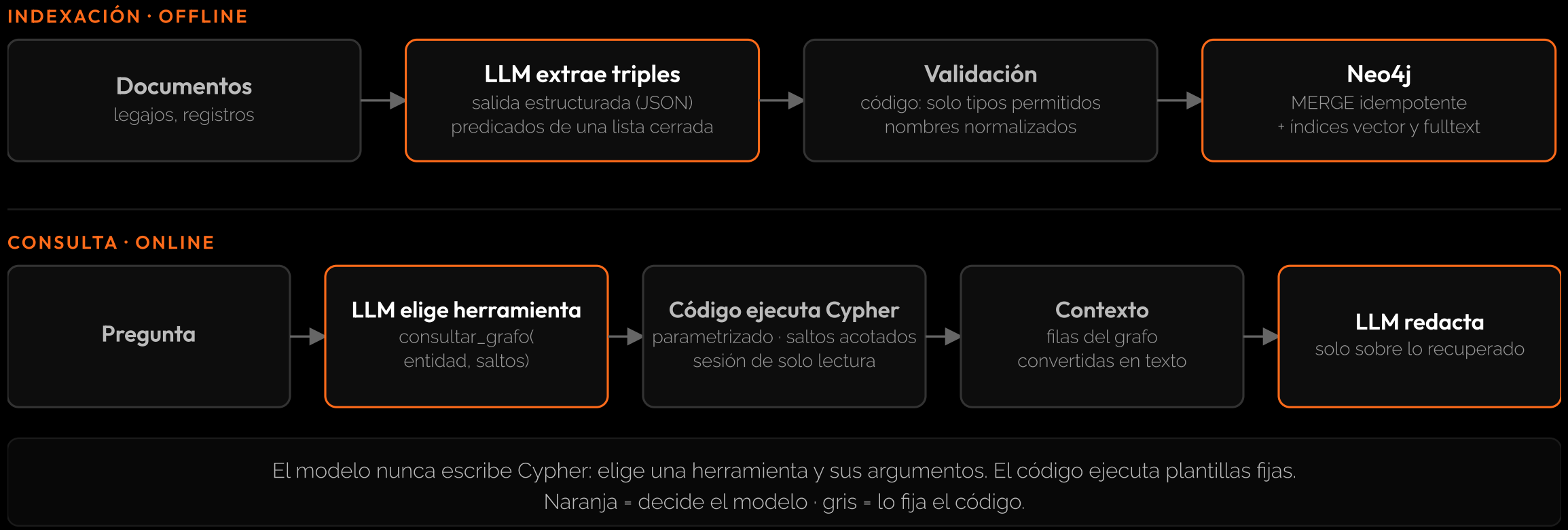
Parámetros (`$s` , `$o`)

Los valores viajan aparte de la consulta: evitan inyección y permiten reutilizarla.

Tipos de relación

No van como parámetro: el código arma una consulta por cada tipo, tomado de una tabla fija (el esquema).

Arquitectura — del texto al grafo y del grafo a la respuesta



El modelo nunca escribe Cypher: elige una herramienta y sus argumentos. El código ejecuta plantillas fijas.
Naranja = decide el modelo · gris = lo fija el código.

La extracción usa un esquema cerrado (los predicados permitidos) y se mide contra ejemplos correctos: precisión y *recall* de los triples. Las lecturas usan una sesión de solo lectura; las escrituras, una aparte.

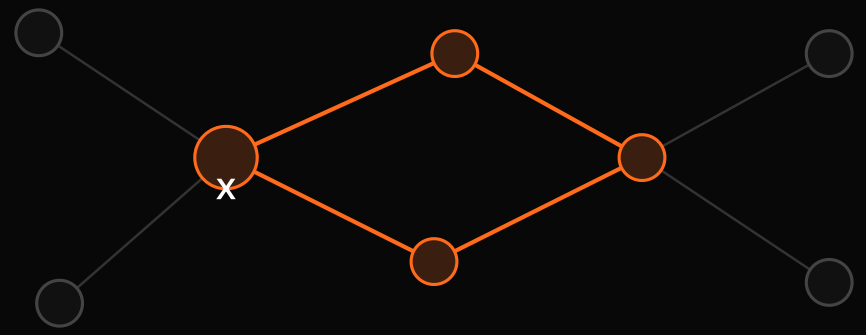
¿Cómo consulta el agente el grafo? Dos enfoques

	TEXTO A CYPHER	HERRAMIENTAS CON CYPHER PARAMETRIZADO
Quién escribe la consulta	El LLM genera Cypher libre	El código: plantillas fijas; el LLM elige herramienta y argumentos
Flexibilidad	Alta: puede expresar casi cualquier pregunta	Acotada a las herramientas definidas
Riesgos	Consultas sin tope, errores de sintaxis, inyección, escrituras accidentales	Bajos: el modelo no puede salirse de las plantillas
Previsibilidad y auditoría	Baja: cada corrida puede generar otra consulta	Alta: se conoce cada consulta posible
Cuándo conviene	Exploración por analistas, con solo lectura y revisión	Producción y entornos regulados

Reglas de las demos: el modelo nunca escribe Cypher · consultas de solo lectura · lo único que se interpola es un entero acotado por código (los saltos) · el nombre de la entidad viaja siempre como parámetro · las sumas las calcula el código.

GraphRAG — búsqueda local vs. búsqueda global

BÚSQUEDA LOCAL · PARTE DE UNA ENTIDAD

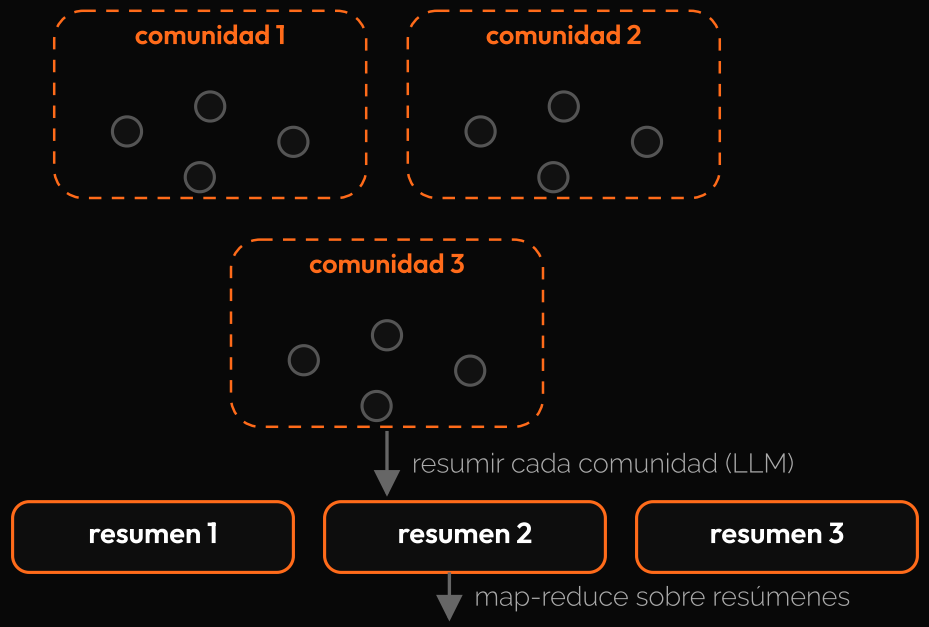


«¿Quién controla a X y con qué exposición?»

Recorre el vecindario (1–3 saltos) de las entidades de la pregunta y pasa ese subgrafo, como texto, al modelo.

Bueno para: relaciones y cadenas concretas.

BÚSQUEDA GLOBAL · PREGUNTA SOBRE EL CORPUS



«¿Cuáles son los temas principales del corpus?»

Bueno para: síntesis global (Microsoft GraphRAG).

El mismo grafo, con estructura societaria

Mismo `build_graph()`, mismo recorrido en anchura (BFS, *Breadth-First Search*). Cambian los datos, no el código.

Triples societarias

```
Textil Norte — controlada_por → Grupo Andes
Grupo Andes — controlada_por → Andes Capital B.V.
Andes Capital B.V. — beneficiario_final_es → M. Ferrari
M. Ferrari — director_en → Agroinsumos del Litoral
```

Para qué

Límite de asistencia crediticia
Partes vinculadas
AML / KYC — beneficiario final

> *"¿Quién es el beneficiario final de Textil Norte y qué exposición consolidada tiene su grupo económico con la entidad?"*

Con `max_hops=3`: 6 aristas → USD 4.6M. Con `max_hops=5`: 8 aristas → USD 5.4M. Sin error ni advertencia. `max_hops` es un parámetro de riesgo, no de performance.

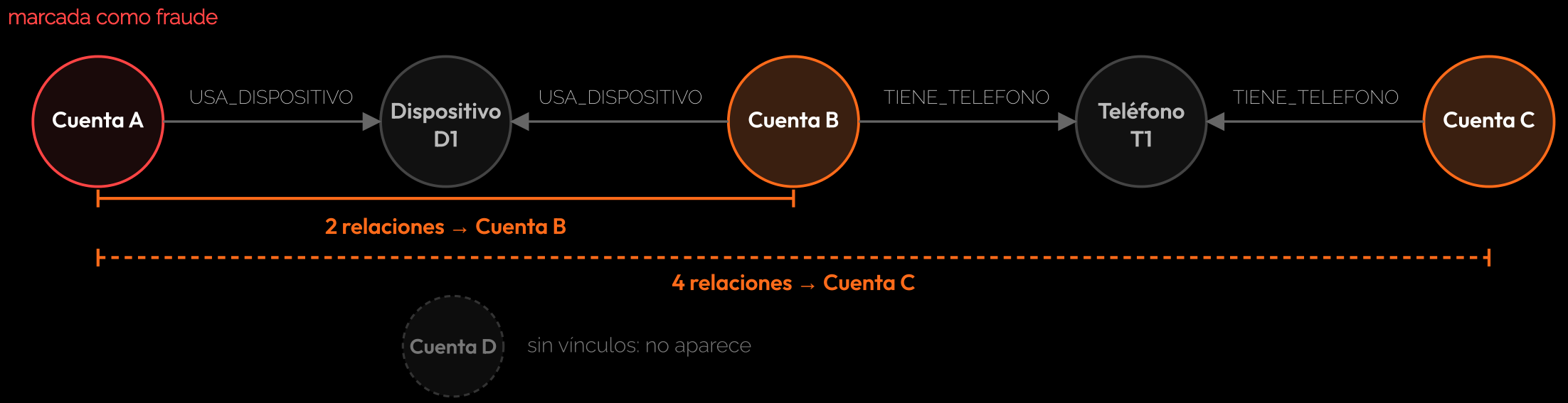
Recorrido multi-salto y el costo de cortar el grafo



Lo que queda más allá del límite no da error ni advertencia: max_hops es un parámetro de riesgo.

Cadena tomada de las triples societarias de la slide anterior; los importes son los de ese ejemplo ilustrativo. Un RAG por similitud no reconstruye esta cadena.

Redes de fraude — atributos compartidos




```
MATCH (a:Cuenta {id: $marcada})
-[:USA_DISPOSITIVO|TIENE_TELEFONO*2..4]-(otra:Cuenta)
WHERE otra <> a
RETURN DISTINCT otra.id
```

El vínculo no está escrito en ninguna parte: emerge de compartir dispositivo o teléfono, con una profundidad que no se conoce de antemano. Mismo patrón para lavado de activos (AML) y dependencias.

Normas que reemplazan y modifican a otras

Años y nombres ilustrativos.



Los tres textos hablan del mismo tema y se parecen entre sí: un RAG puede devolver la v1 o la v2 con igual confianza.

```
MATCH (n:Norma {tema: 'límite de crédito'})
WHERE NOT EXISTS { (n)<-[:REEMPLAZA]-(:Norma) }
RETURN n.titulo, n.fecha
```

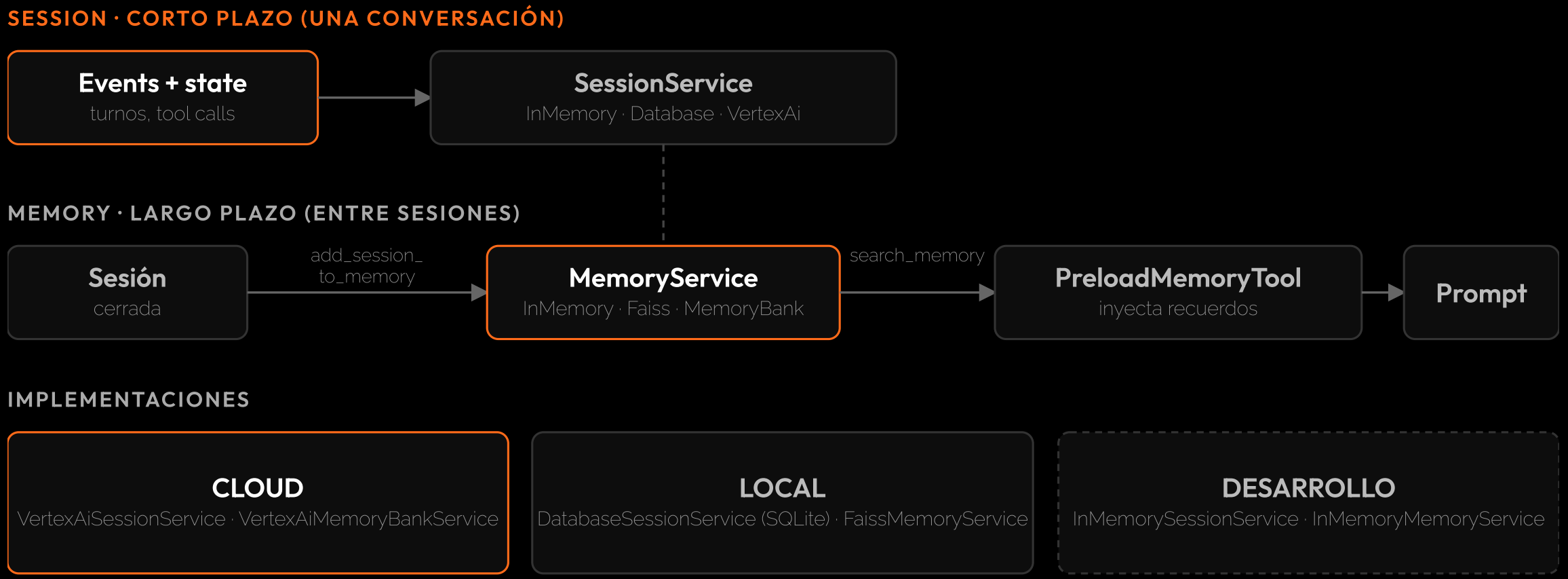
La pregunta «¿cuál está vigente?» es una ausencia: la versión vigente es la que *ninguna otra* reemplaza. Un RAG por similitud no puede demostrar que algo no existe; un grafo lo expresa con una condición de no existencia.

Comparación — RAG chunked vs. GraphRAG

DIMENSIÓN	RAG CHUNKED	GRAPHRAG
Preguntas simples	Excelente	Overkill
Multi-hop (3+ saltos)	Falla	Excelente
Tiempo de indexación	Minutos	Horas–días
Mantenimiento	Bajo	Alto
Actualización en tiempo real	Re-embed	Validación en grafo
Herramienta	FAISS / Vertex AI	Neo4j / Neptune
Costo operativo	Bajo	Significativamente mayor

Regla: usar GraphRAG solo cuando el RAG chunked falla en producción. Productionizar un grafo de conocimiento dinámico es un proyecto de ingeniería mayor.

Sesión vs. memoria en ADK2



La Session vive una conversación; la Memory se alimenta al cerrarla y se consulta en sesiones futuras. La interfaz es la misma en las tres columnas.

ADK2 — sesión, estado y Runner

SERVICIO DE SESIÓN	PERSISTENCIA	USO
`InMemorySessionService`	Ninguna (se pierde al reiniciar)	Desarrollo y pruebas
`DatabaseSessionService`	Base SQL (p. ej. SQLite)	Local o simple
`VertexAiSessionService`	Gestionada por Vertex AI	Producción

```
await svc.create_session(app_name="tutor",
    user_id="sofia", session_id="s1")
runner = Runner(
    agent=agent, app_name="tutor",
    session_service=svc)
async for ev in runner.run_async(
        user_id="sofia",
        session_id="s1",
        new_message=msg):
    if ev.is_final_response(): ...
```

Una sesión se identifica por `app_name` + `user_id` + `session_id` y contiene eventos (turnos, llamadas a tools) y estado (clave–valor). El `Runner` reinyecta ese historial en cada llamada al modelo.

> `run_async` es un generador asíncrono de eventos. El `Runner` no crea la sesión por sí solo: si no existe, falla con `SessionNotFoundError`. Se evita con un `create_session` previo (como arriba) o con el atajo `auto_create_session=True`.

ADK2 — memoria de largo plazo: escribir, leer, inyectar

Escribir

`add_session_to_memory(session)`

Al cerrar la sesión, se envía a la memoria de largo plazo. Qué persistir es una decisión de diseño.

Leer

`search_memory(app_name, user_id, query)`

Consulta la memoria y devuelve los recuerdos al código. No toca el prompt. Siempre acotada al usuario.

Inyectar

`PreloadMemoryTool()`

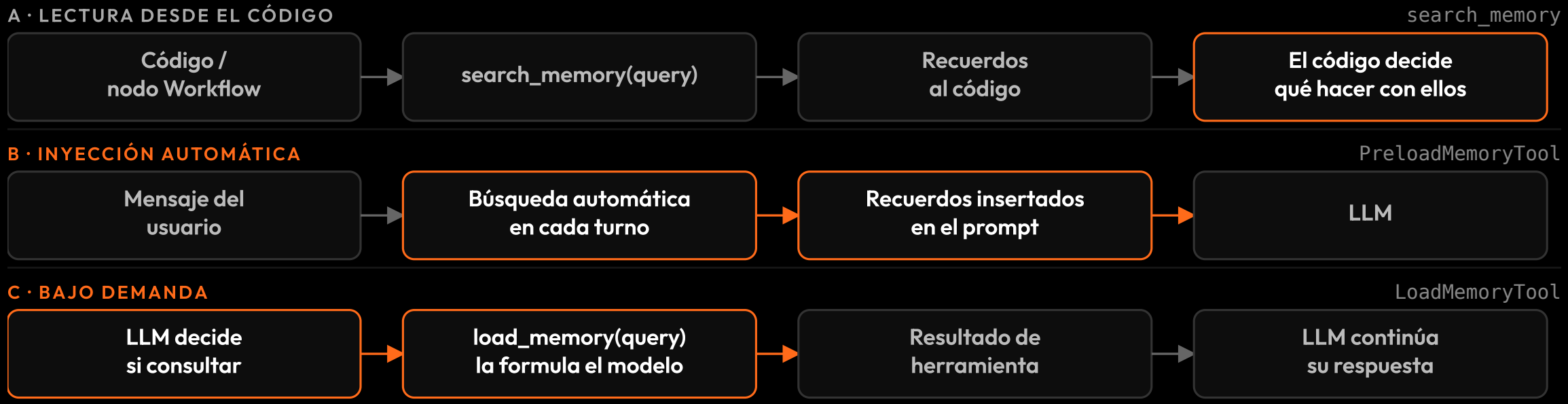
Es una lectura automática en cada turno: busca con el mensaje del usuario y agrega los recuerdos al prompt. El modelo no decide.

Leer e inyectar no son pasos paralelos: inyectar = leer + insertar en el prompt, automáticamente. Cuándo conviene cada uno: slide 53.

	NUBE (GESTIONADO)	LOCAL (PROPIO)
Sesión	`VertexAiSessionService`	`DatabaseSessionService` (SQLite)
Memoria	`VertexAiMemoryBankService` (requiere un `agent_engine_id`)	Subclase de `BaseMemoryService` sobre FAISS, con aislamiento por (app, usuario)

El agente depende de la interfaz, no de la implementación. En la nube, `location` decide dónde quedan las conversaciones (Slide 65).

Leer, inyectar o dejar que el modelo decida



MECANISMO	QUIÉN DECIDE	CONVIENE CUANDO	RIESGO
`search_memory` (código)	El código	Se necesita control: filtrar, mostrar, medir, nodo de un `Workflow`	Hay que escribir la lógica que lo lleva (o no) al prompt
`PreloadMemoryTool`	Automático, cada turno	Casi todos los turnos se benefician (tutor, asistente) y los mensajes se entienden solos	Busca con el mensaje crudo («¿y el otro?» recupera poco); puede agregar ruido
`LoadMemoryTool`	El modelo	La memoria sirve solo a veces; el modelo puede reformular la consulta	Puede no consultarla cuando debía

Mapa de equivalencias: cloud y local

CAPA	CLOUD (GCP)	LOCAL
Búsqueda vectorial	Vertex AI Vector Search	FAISS + IDSelectorBatch
Embeddings	text-embedding-005	sentence-transformers (multilingüe)
Modelo (Agent)	gemini-2.5-flash	gemini-2.5-flash (igual)
Sesión	VertexAiSessionService	DatabaseSessionService (SQLite)
Memoria larga	VertexAiMemoryBankService	FaissMemoryService (propio)
Filtro de acceso	restricts	IDSelectorBatch
Dónde vive el dato →	vectores y consultas en GCP (región = compliance)	vectores y sesiones locales; el prompt va a Gemini

Lo que no cambia es el código de ADK2 (Agent, Workflow, Runner); cambian los servicios que se le enchufan.

Qué no hacer — Anti-patterns críticos

Ventana FIFO en sesiones largas — el diagnóstico inicial sale primero del prompt y el agente «olvida» el problema raíz.

GraphRAG para Q&A simple — complejidad enorme e innecesaria; BM25 o vectores resuelven la gran mayoría.

Grafos sin validación — una entidad incorrecta corrompe todas sus relaciones.

Embeddings obsoletos — si cambia el modelo o el corpus, hay que reindexar: los vectores viejos no son comparables con los nuevos.

Agentic RAG sin topes — bucles, costo impredecible y respuestas que cambian entre corridas.

ACL en el prompt y no en el retrieval — si el documento entró al contexto, ya se filtró tarde.

Saltos sin dueño ni aviso — un `max_hops` elegido como perilla de latencia cambia el resultado de negocio sin error.

Dejar que el LLM escriba Cypher libre — consultas sin tope, errores e inyección; en producción, herramientas parametrizadas.

Conceptos clave — Recuperación

TÉRMINO	DEFINICIÓN
Context window	Tokens que se pasan al modelo en una llamada; memoria de trabajo de esa request
BM25	Ranking léxico: frecuencia de término × IDF, normalizado por longitud
Índice invertido	Mapea cada token a los documentos que lo contienen
Embedding	Vector denso que captura significado; consulta y documentos, con el mismo modelo
ANN (FAISS, HNSW, IVF)	Vecinos más cercanos aproximados: velocidad a cambio de exactitud
Chunking	Cortar documentos en fragmentos; define qué se puede encontrar
RRF	Fusión por rangos: suma 1/(k + posición) de cada ranking; no normaliza puntajes
Bi-encoder / cross-encoder	Codifica por separado (rápido) / evalúa el par junto (preciso, lento)
HyDE	Buscar con el embedding de una respuesta hipotética, no el de la pregunta

Conceptos clave — Memoria, grafos y agentes

TÉRMINO	DEFINICIÓN
Memoria episódica / semántica / procedural	Experiencias concretas / hechos generales / cómo hacer las cosas
Knowledge graph	Entidades (nodos) y relaciones (aristas) con tipos y propiedades
Property graph	Modelo de grafo donde nodos y relaciones llevan etiqueta o tipo y propiedades (Neo4j)
Esquema	Qué etiquetas, tipos de relación y propiedades son válidos en un grafo
Entity resolution	Unificar menciones de la misma entidad entre fuentes
Cypher	Lenguaje declarativo de consulta de grafos de Neo4j (no es una sigla)
Community summaries	Resúmenes por comunidad del grafo; sostienen la búsqueda global
Agentic RAG	El agente decide si, dónde y cuántas veces recuperar, con topes en código
Reranking	Reordenar un top-k amplio con un modelo más preciso antes de generar

Siglas — recuperación y búsqueda

SIGLA	SIGNIFICA	EN UNA FRASE
RAG	*Retrieval-Augmented Generation*	Generar una respuesta apoyándose en información recuperada de una fuente
LLM	*Large Language Model*	Modelo de lenguaje de gran tamaño (Gemini, GPT, Claude…)
BM25	*Best Matching 25*	Ranking léxico de documentos por coincidencia de términos
TF-IDF	*Term Frequency – Inverse Document Frequency*	Pondera un término por lo frecuente en el documento y lo raro en el corpus
ANN	*Approximate Nearest Neighbors*	Vecinos más cercanos aproximados: rápido, con pequeña pérdida de exactitud
FAISS	*Facebook AI Similarity Search*	Biblioteca de búsqueda de similitud entre vectores
IVF	*Inverted File*	Índice que agrupa vectores en celdas y busca solo en las cercanas
HNSW	*Hierarchical Navigable Small World*	Índice de grafo de vecinos en capas navegables
ScaNN	*Scalable Nearest Neighbors*	Búsqueda aproximada de Google; base de Vertex AI Vector Search
RRF	*Reciprocal Rank Fusion*	Fusiona rankings sumando 1/(k + posición); no normaliza puntajes
HyDE	*Hypothetical Document Embeddings*	Buscar con el embedding de una respuesta hipotética

Siglas — memoria, seguridad y plataforma

SIGLA	SIGNIFICA	EN UNA FRASE
FIFO	*First In, First Out*	El primero que entra es el primero que sale
TTL	*Time To Live*	Tiempo de vida de un dato antes de expirar
ACL	*Access Control List*	Lista de usuarios, grupos o roles autorizados a un recurso
PII	*Personally Identifiable Information*	Datos que permiten identificar a una persona
AML	*Anti-Money Laundering*	Prevención del lavado de activos
KYC	*Know Your Customer*	Conocimiento del cliente: verificación de identidad y beneficiario final
ADK	*Agent Development Kit*	Framework de Google para construir agentes (aquí, ADK2)
GCP	*Google Cloud Platform*	La nube de Google
ReAct	*Reasoning + Acting*	Patrón que alterna razonamiento y acción
CoALA	*Cognitive Architectures for Language Agents*	Marco que organiza la memoria y la acción de agentes con lenguaje
API	*Application Programming Interface*	Interfaz para que un programa use a otro

Siglas — grafos y datos

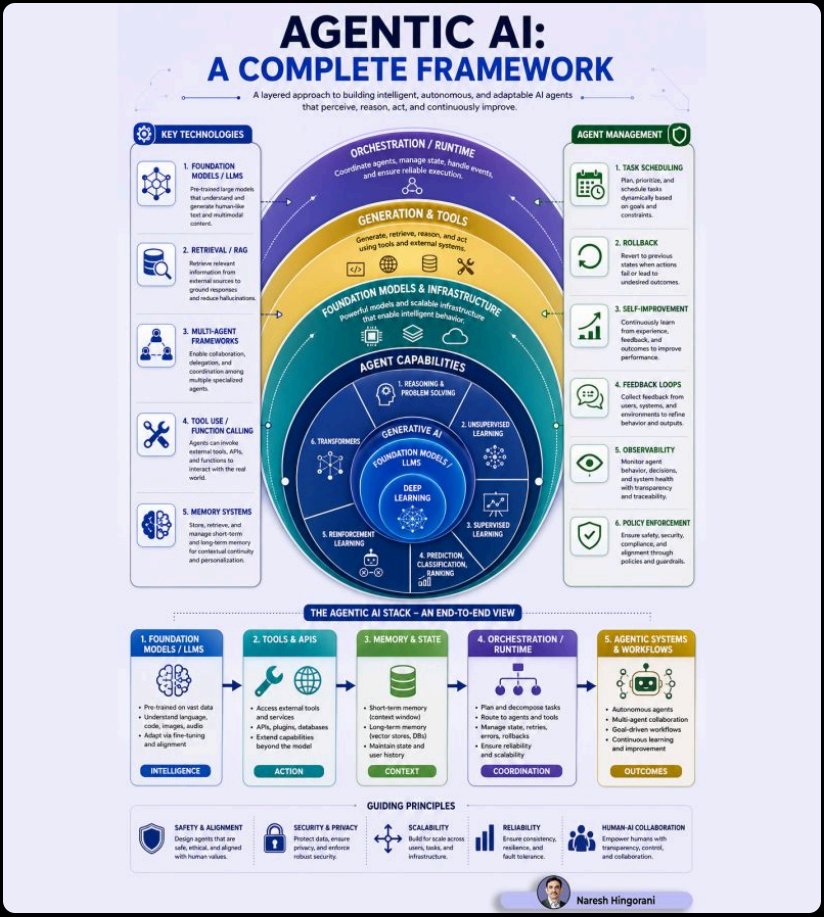
SIGLA	SIGNIFICA	EN UNA FRASE
KG	*Knowledge Graph*	Grafo de conocimiento
NER	*Named Entity Recognition*	Reconocer entidades con nombre (personas, empresas) en un texto
BFS	*Breadth-First Search*	Búsqueda en anchura: explora todos los vecinos antes de avanzar un nivel
SQL	*Structured Query Language*	Lenguaje de consulta de bases relacionales
ACID	*Atomicity, Consistency, Isolation, Durability*	Garantías de una transacción confiable
RDF	*Resource Description Framework*	Modelo de datos basado en triples sujeto – predicado – objeto
RDFS	*RDF Schema*	Vocabulario para describir clases y propiedades en RDF
OWL	*Web Ontology Language*	Lenguaje de ontologías con semántica formal
SPARQL	*SPARQL Protocol and RDF Query Language*	Lenguaje de consulta de RDF (sigla recursiva)
GQL	*Graph Query Language*	Estándar ISO para consultar property graphs, cercano a Cypher
W3C	*World Wide Web Consortium*	Organismo que define estándares web, entre ellos RDF y OWL

Knowledge + Memory — vista de arquitectura



El modelo solo procesa lo que recibe. La selección, la recuperación, la evaluación y los controles viven en la infraestructura que lo rodea.

Agentic AI: un marco completo — dónde cae el Módulo 6



Infografía: Naresh Hingorani

3.ª Memory & State — La capa del stack donde vive M6: memoria de corto plazo (ventana de contexto) y de largo plazo (vector stores, DBs).

• Key technologies — «Retrieval / RAG» y «Memory systems» son dos de las cinco tecnologías clave: las dos del módulo.

= Sesión vs. memoria — Corto plazo = `Session`, largo plazo = `Memory` de ADK2 (ver Slide 50).

→ Agent management — Rollback, observabilidad, policy enforcement: fuera del alcance de M6.

Ojo: es una infografía genérica y no la usaría como referencia técnica (por ejemplo, pone «Transformers» como una capacidad del agente). Sirve para situar el módulo, no para citar.

Módulo 6 en el contexto del curso

MÓDULO 2

Memoria de corto y largo plazo introducida — aquí se profundiza.

MÓDULO 5

Ingeniería de contexto: memoria y conocimiento alimentan el ensamblado del prompt.

MÓDULO 6 ← ESTAMOS AQUÍ

RAG, GraphRAG, jerarquía de memoria, ADK2.

MÓDULO 7

Aprendizaje: la memoria experiencial es la base del aprendizaje del agente.

MÓDULO 9

Evaluación de sistemas de memoria: precisión de recall, métricas de frescura.

LABS

RAG con Vertex AI Vector Search + ADK2 Session Memory API.

6 Takeaways fundamentales

1. Memoria: cuatro tipos (working, episódica, semántica, procedural), un ciclo (escribir, leer, actualizar, olvidar) y cuatro niveles de mecanismo. FIFO lo aplica la aplicación al armar el prompt. Elegir el nivel más bajo que funcione.

2. RAG moderno: chunking, metadata con ACL, índice híbrido con RRF y reranking mejoran más que cambiar de modelo. Medir antes de agregar complejidad.

3. Agentic RAG: el modelo decide qué y cuándo recuperar; el código fija topes, herramientas y presupuesto.

4. Grafos de conocimiento: responden cadenas, agregaciones, caminos, patrones y ausencias que un RAG por fragmentos no resuelve. Exigen esquema, validación y resolución de entidades.

5. Neo4j y Cypher: se dibuja el patrón; `MERGE` para cargar, parámetros para consultar y saltos acotados. En producción, herramientas parametrizadas y el modelo nunca escribe Cypher.

6. ADK2: la interfaz no cambia entre nube y local; cambian los servicios (Vertex AI o FAISS + SQLite). En local, solo el prompt viaja a Gemini.

RAG en entorno regulado — 5 controles

1 · ACL (lista de control de acceso) en el retrieval, no en el prompt — El campo `restricts` (Slide 20) etiqueta cada vector con su nivel de acceso; en local, el equivalente es `IDSelectorBatch`. Si el documento entró al prompt, ya se filtró tarde.

2 · PII (datos personales identificables) antes de embeber — El embedding hereda la clasificación del documento. Y la consulta también es dato sensible: si se embebe afuera, salió del perímetro.

3 · Residencia de datos — `location=` define en qué país viven los vectores. Es compliance disfrazado de argumento de función: migrar de región es reindexar de cero.

4 · Retención y derecho al olvido — Borrar un vector es fácil; *saber cuál* exige trazabilidad documento → chunks → vectores desde el día uno. También en la memoria del agente (Slide 6: «olvidar»).

5 · Trazabilidad de la cita — Los chunks que *entraron* al prompt no son las fuentes. Hay que quedarse con los que el modelo citó.

Los cinco se diseñan en la ingesta. Ninguno se agrega después sin reindexar.

MÓDULO 6 · CIERRE

Un agente con buena memoria es un agente que aprende.

Próximo módulo: Módulo 7 — Aprendizaje y Adaptación

La memoria experiencial de hoy es el fundamento del aprendizaje de mañana.

LAB DISPONIBLE EN /MODULO_06/LAB.MD